

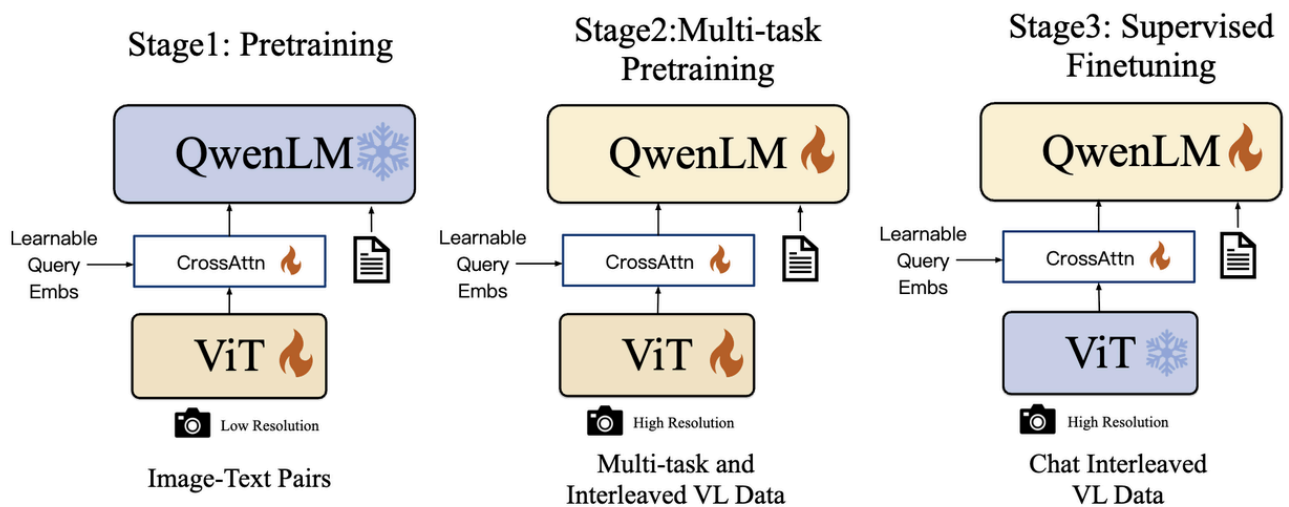
[Qwen-VL](#)是比较早期的开源多模态大模型，在[Qwen2-VL](#)、[Qwen2.5-VL](#)、[Qwen3-VL](#) ([Qwen3 VL Preview](#))、[Qwen-3.5 Preview](#)的几代改进中，也能够窥探到自23年以来多模态大模型的发展路径。

同时，[Qwen-VL](#)系列作为开源多模态中比较知名的存在，如果面试者在项目中使用过，也常常被面试官提问“Qwen-VL每个版本之间的改动？各自的关键技术？”

本期就来介绍[Qwen-VL](#)、[Qwen2-VL](#)、[Qwen2.5-VL](#)、[Qwen3-VL](#) ([Qwen3 VL Preview](#))、[Qwen-3.5 Preview](#)各个版本之间的架构和训练策略的对比。

1. Qwen-VL

1.1 架构



- **ViT**: ViT架构和预训练权重来自于 [OpenCLIP](#) 的ViT-bigG模型，**未作任何修改** ([OpenCLIP](#) ViT-bigG/14训练使用LAION-2B数据，以及采样自[LAION-5B](#)的英文标注图文pair数据)。并且，ViT-bigG使用的是**可学习的绝对位置编码 (absolute positional embeddings)**，不是 RoPE (Rotary Position Embedding)
- **Projector**: 一个随机初始化的**单层交叉注意力模块** (single-layer cross-attention module initialized randomly)。此模块使用一组可训练的向量（嵌入）作为query，使用来自视觉编码器的图像特征作为交叉注意力操作的keys、values。这个机制将视觉特征序列压缩到固定长度的256。**并且在Projector模块中使用2D绝对位置编码**，以保证LLM侧的位置编码不做改动。

```
class Resampler(nn.Module):
    """
    A 2D perceiver-resampler network with one cross attention layers by
    (grid_size**2) learnable queries and 2d sincos pos_emb
    Outputs:
    A tensor with the shape of (grid_size**2, embed_dim)
    """
    ...
    self.num_queries = grid_size ** 2
    self.embed_dim = embed_dim
    self.num_heads = num_heads
    # 这就是2D的位置编码
    self.pos_embed = nn.Parameter(
        torch.from_numpy(get_2d_sincos_pos_embed(embed_dim, grid_size)).float()
    ).requires_grad_(False)
    # 就是256的长度
```

```

self.query = nn.Parameter(torch.zeros(self.num_queries, embed_dim)
...

def forward(self, x, attn_mask=None):

    pos_embed = get_abs_pos(self.pos_embed, x.size(1))

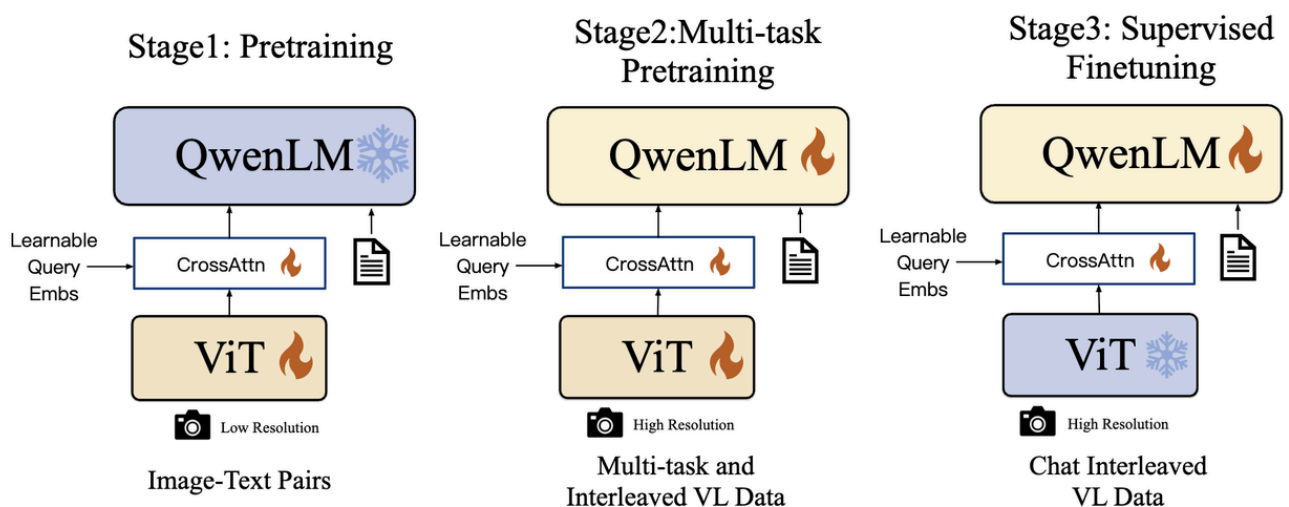
    x = self.kv_proj(x)
    x = self.ln_kv(x).permute(1, 0, 2)

    N = x.shape[1]
    q = self.ln_q(self.query)
    out = self.attn(
        self._repeat(q, N) + self.pos_embed.unsqueeze(1),
        x + pos_embed.unsqueeze(1),
        x,
        attn_mask=attn_mask)[0]
    return out.permute(1, 0, 2)
...

```

- **LLM**: Qwen-VL的大型语言模型使用了来自Qwen-7B模型的预训练权重。

1.2 训练&数据



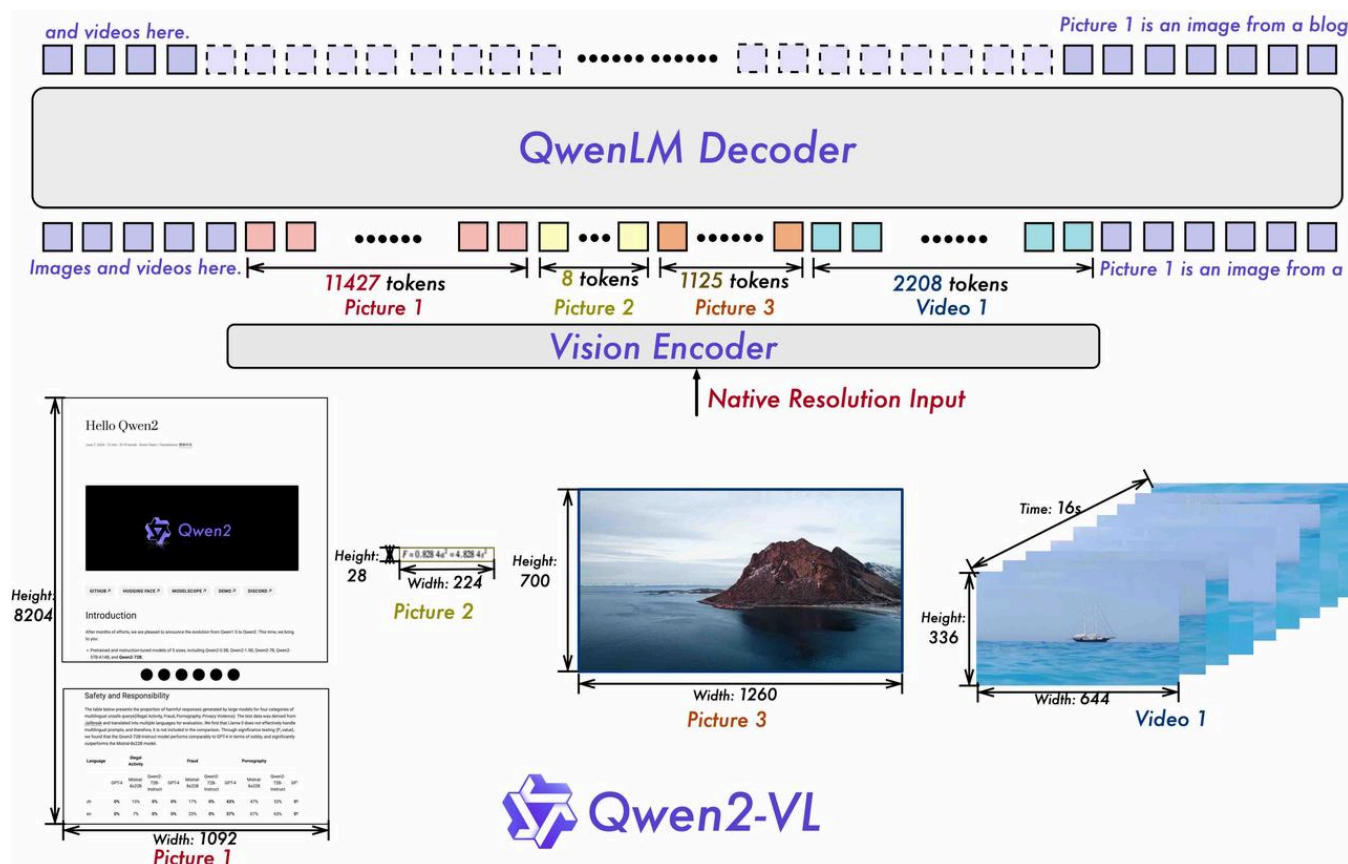
- Stage1 为**对齐预训练**，使用大量的图文Pair对数据对齐视觉模块和LLM的特征，这个阶段冻结LLM模块的参数；
- Stage2 为**多任务预训练**，使用更高质量的图文多任务数据（主要来源于开源VL任务，部分自建数据集），**更高的图片像素输入**，全参数训练；
- Stage3 为**指令微调**，这个阶段冻结视觉Encoder模块，使用的数据主要来自大模型Self-Instruction方式自动生成，目标是提升模型的指令遵循和多轮对话能力。

2. Qwen2-VL

[Qwen2-VL](#) 相比 [Qwen-VL](#) 的架构改进关注在：

- 采用了 [LLaVA\(2023, Microsoft\)](#) 的 MLP Projector 连接，并且使用效率更优的 Patch Merger。
- 视觉优先的思想，重新训练了 [NaViT](#) 的视觉编码器，并且使用了 2D-RoPE
- 相比于 [Qwen-VL](#)，多模态位置编码从 Projector 迁移到 LLM，并且使用 [MRoPE](#)

2.1 架构



- **ViT**: 重新训练了675M的ViT，并且使用了 [NaViT](#) 和 2D-RoPE，支持动态分辨率。
- **Projector**: Qwen [Qwen2-VL](#) 实质上用了两层MLP作为Adaptor，这个模块PatchMerger被放在了VE也就是 Qwen2VisionTransformerPretrainedModel 里面了，也就是大名鼎鼎的 PatchMerger：

```
class PatchMerger(nn.Module):
    def __init__(self, dim: int, context_dim: int, spatial_merge_size: int = 2) -> None:
        super().__init__()
        self.hidden_size = context_dim * (spatial_merge_size**2)
        self.ln_q = LayerNorm(context_dim, eps=1e-6)
        self.mlp = nn.Sequential(
            nn.Linear(self.hidden_size, self.hidden_size),
            nn.GELU(),
            nn.Linear(self.hidden_size, dim),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.mlp(self.ln_q(x).view(-1, self.hidden_size))
        return x
```

- **LLM**：文本模型初始化于Qwen2, 不同参数量LLM模型对应同一个ViT。并且LLM的位置编码替换为MRoPE，简单理解就是pos embedding从hidden_dim上进行了分割，分别来表示time、height、width，具体代码实现参考 [MRoPE](#)。

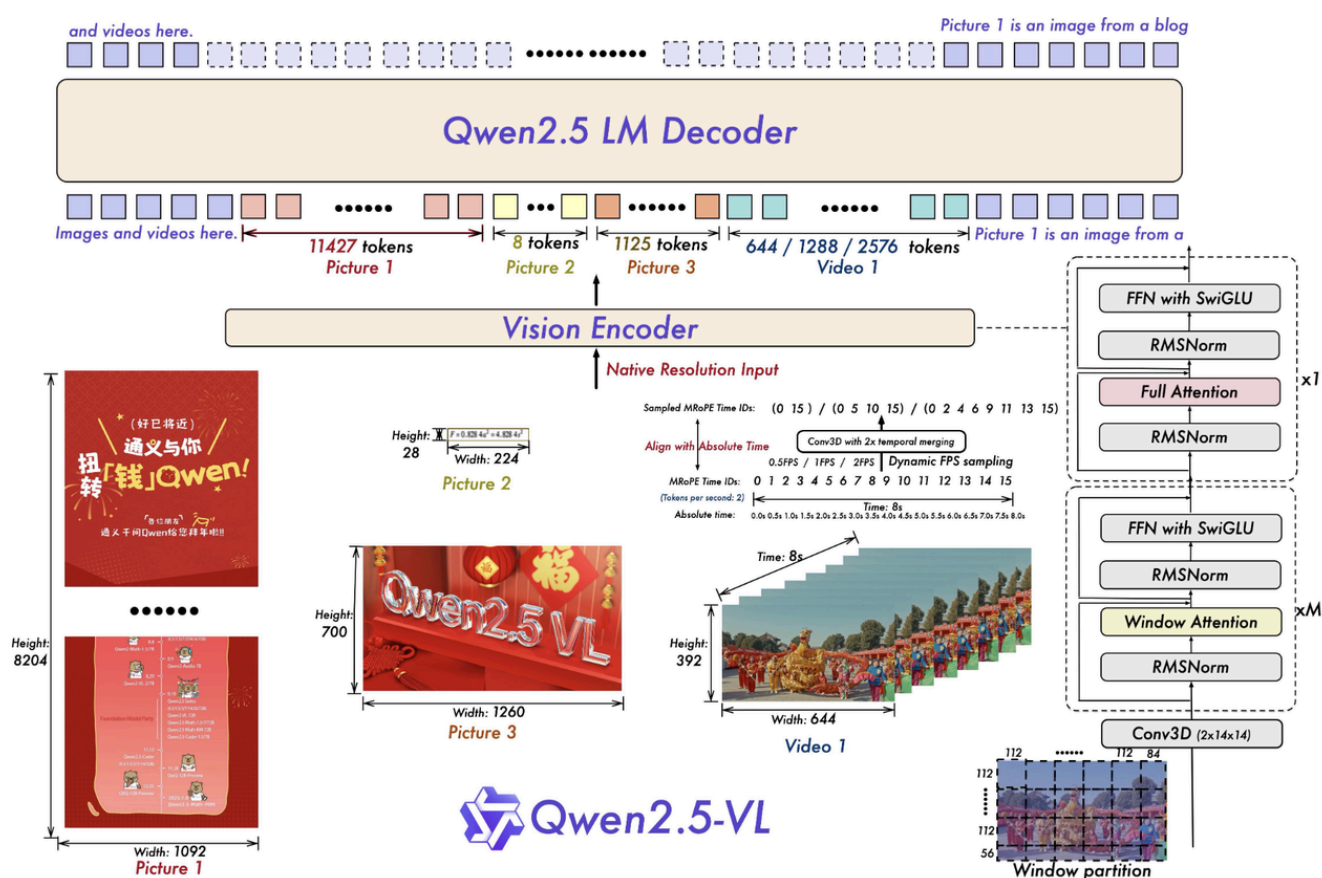
2.2 训练

三阶段训练：

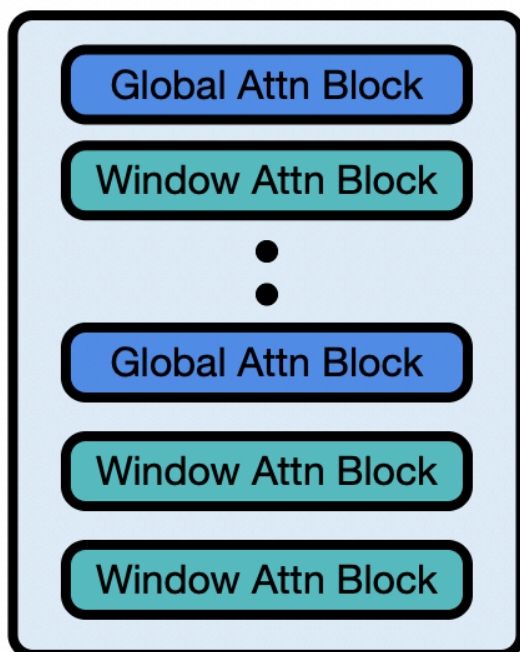
- Stage 1: 利用图像-文本对训练**视觉编码器和连接层**。使用了 600B 的token（图像分类和OCR数据）。
- Stage 2: **训练所有参数**。使用 800B token。这一阶段引入更高质量的混合图像-文本内容，包含多种任务和纯文本数据。
- Stage 3: **只训练语言模型**。数据包含多种任务instructional datasets、agent训练数据。

3. Qwen2.5-VL

3.1 架构



Qwen-2.5



[Qwen2.5-VL](#)在[Qwen2-VL](#)上的改进点：

- **ViT：使用了更接近LLM的架构：**
 - Attention结构上使用了交错的 **Window Attention**。
 - 使用SwiGLU的FFN结构，`hidden_act="silu"`（参考[1.1.5 前馈网络 FFN 和激活函数 Activation](#)）并且增加**bias**，注意这里在llama这样的语言模型中是默认无偏置。
 - 标准化使用 RMSNorm。
- **Projector：**仍然使用的是PatchMerger，但是**替换了LayerNorm为RMSNorm**：

```
class Qwen2_5_VLPatchMerger(PatchMerger):
    def __init__(self, dim: int, context_dim: int, spatial_merge_size: int = 2) -> None:
        super().__init__(dim, context_dim, spatial_merge_size)
        self.ln_q = Qwen2RMSNorm(context_dim, eps=1e-6)

#####
class PatchMerger(nn.Module):
    def __init__(self, dim: int, context_dim: int, spatial_merge_size: int = 2) -> None:
        super().__init__()
        self.hidden_size = context_dim * (spatial_merge_size**2)
```

```
self.ln_q = LayerNorm(context_dim, eps=1e-6)
self.mlp = nn.Sequential(
    nn.Linear(self.hidden_size, self.hidden_size),
    nn.GELU(),
    nn.Linear(self.hidden_size, dim),
)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    x = self.mlp(self.ln_q(x).view(-1, self.hidden_size))
    return x
```

- **LLM**：语言模型升级使用了 Qwen2.5。MRoPE（在视频输入）对齐了现实video的帧率，即**对齐了绝对时间**。

具体代码解读参考 [Qwen2.5-VL](#)。

3.2 训练

Stages	Visual Pre-Training	Multimodal Pre-Training	Long-Context Pre-Training
Data	Image Caption Knowledge OCR	+ Pure text Interleaved Data VQA, Video Grounding, Agent	+ Long Video Long Agent Long Document
Tokens	1.5T	2T	0.6T
Sequence length	8192	8192	32768
Training	ViT	ViT & LLM	ViT & LLM

Table 2: Training data volume and composition across different stages.

权重初始化：LLM 的权重直接初始化于 Qwen2.5 LLM，ViT使用 DataComp 从 0 进行训练（Stage 0）。

- Stage 1:

训练ViT部分+PatchMerger，数据包括 Image Caption、Knowledge、OCR，使视觉部分与文本数据进行对齐。

- Stage 2:

整个模型（包括ViT、PatchMerger、LLM）一起进行训练，使用了更多样的多模态数据，如视频数据、视觉问答等，以提升模型的多模态处理能力。

- Stage 3:

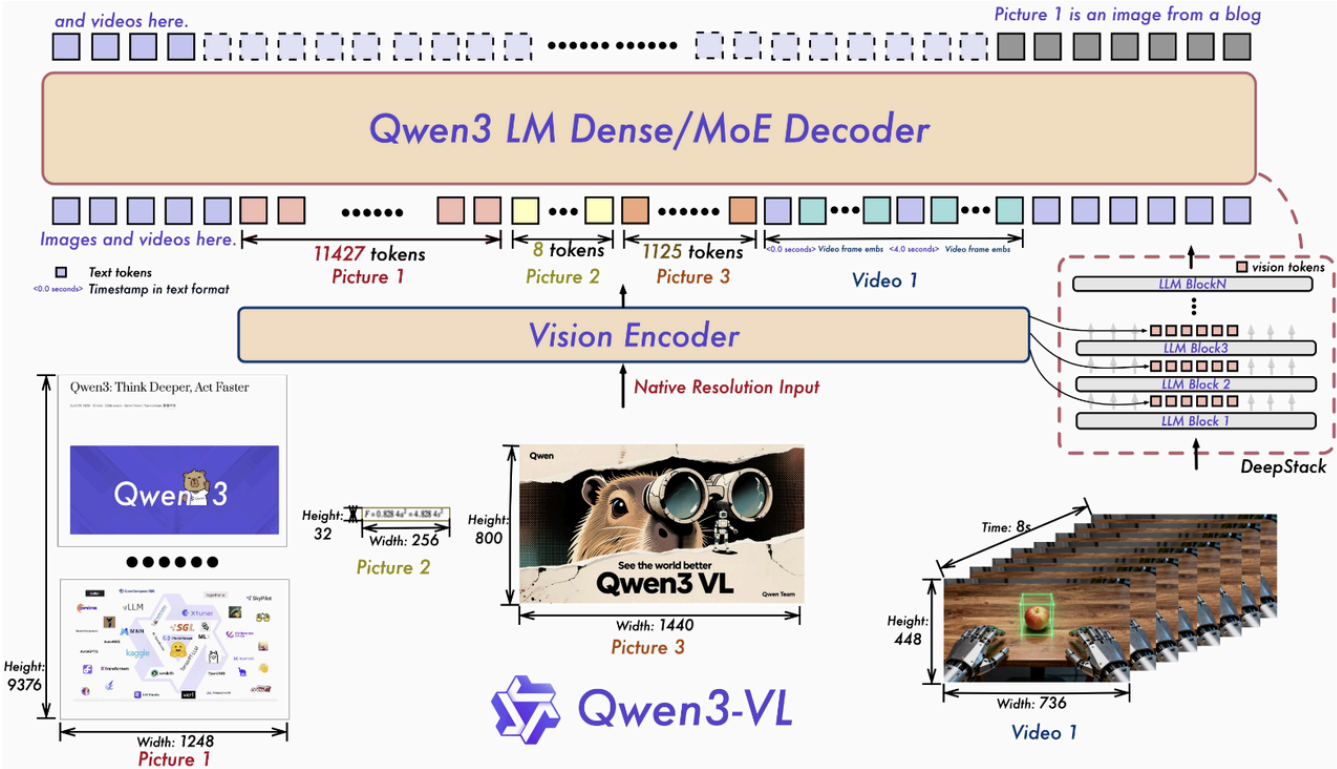
整个模型（包括ViT、PatchMerger、LLM）一起进行训练，第三阶段通过增加序列长度和加入视频与代理数据，使模型能够处理更长的上下文和更复杂的推理任务。

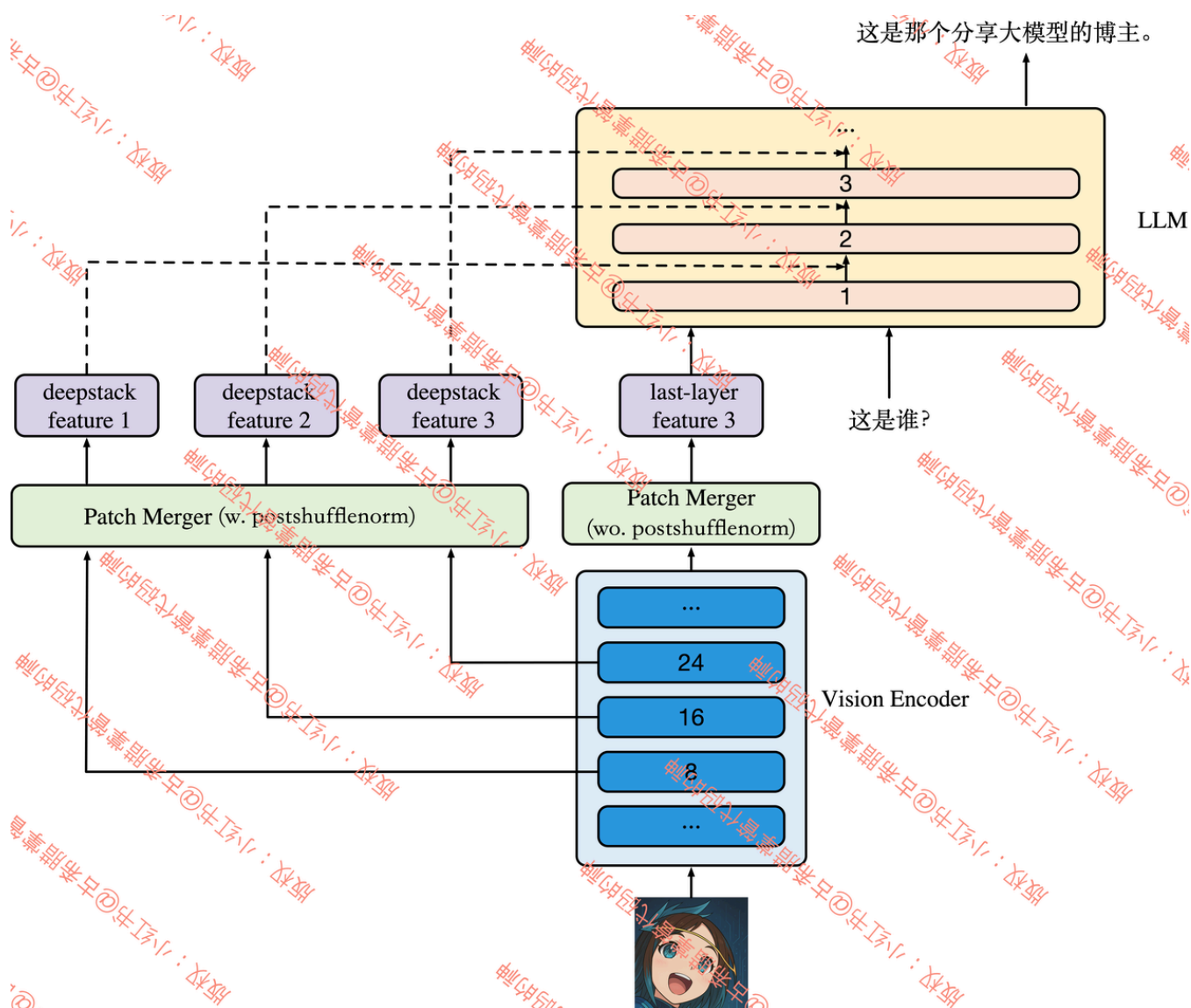
[Qwen2.5-VL](#)在 [Qwen2-VL](#) 上训练的关键改良：**增加后训练**

在Qwen2.5-VL的后训练（微调）阶段，模型采用了**监督微调（SFT）**和**直接偏好优化（DPO）**这两种方法来进一步优化其在特定任务中的表现，**均保持ViT冻结**。

4. Qwen3-VL

4.1 架构





- ViT: 沿用 Qwen2.5 bias 配置、[2D-RoPE](#)，但是替换了激活函数，PatchEmbed启用bias。
- Projector: Adaptor本次更新中使用了两个部分特征，一部分是沿用的Qwen2.5的 PatchMerger（对比解析参考 [MLLM 之 MLP Projector 发展史](#)）另外一部分使用了 `deepstack_visual_indexes`，用于与LLM特征进行融合。
- LLM: [Qwen3-VL-235B-A22B](#) 使用的是 [Qwen 3](#) 的MoE版本（还会有非MoE版本开源），架构上使用了QKNorm（解析参考 [QK Norm 的前世今生](#)），Qwen3VL 使用 3 维多模态位置编码 [MRoPE](#)，并且采用了MRoPE-Interleave，t,h,w 交错分布实现对时间，高度和宽度的全频率覆盖，提升鲁棒性和外推性能。在前K层使用了 **DeepStack特征融合**（解析详细参考 [Qwen3-VL的 DeepStack 技术是什么？](#)）

4.2 训练

Stage	Objective	Training	Token Budget	Sequence Length
S0	Vision-Language Alignment	Merger	67B	8,192
S1	Multimodal Pre-Training	All	~1T	8,192
S2	Long-Context Pre-Training	All	~1T	32,768
S3	Ultra-Long-Context Adaptation	All	100B	262,144

首先在S0之前需要对ViT进行CPT（继续预训练），主要目标是拓展其在动态分辨率下的能力。

1. **Stage-0: 视觉-语言对齐**: 仅训练MLP融合器，视觉编码器和LLM冻结，数据：67B tokens（高质量图文对、OCR、世界知识）
2. **Stage-1: 多模态预训练**: 全参数训练，混合视觉Instruction数据与纯文本数据，数据：~1T tokens（含交错图文、视觉定位、VQA、STEM数据及少量视频）
3. **Stage-2: 长上下文预训练**: 序列长度增至32768，增加纯文本比例和视频/任务数据，数据：~1T tokens（强化长视频与多步骤任务理解）
4. **Stage-3: 超长上下文适应**: 序列长度扩展至262144，数据：100B tokens（聚焦长视频/长文档分析任务）

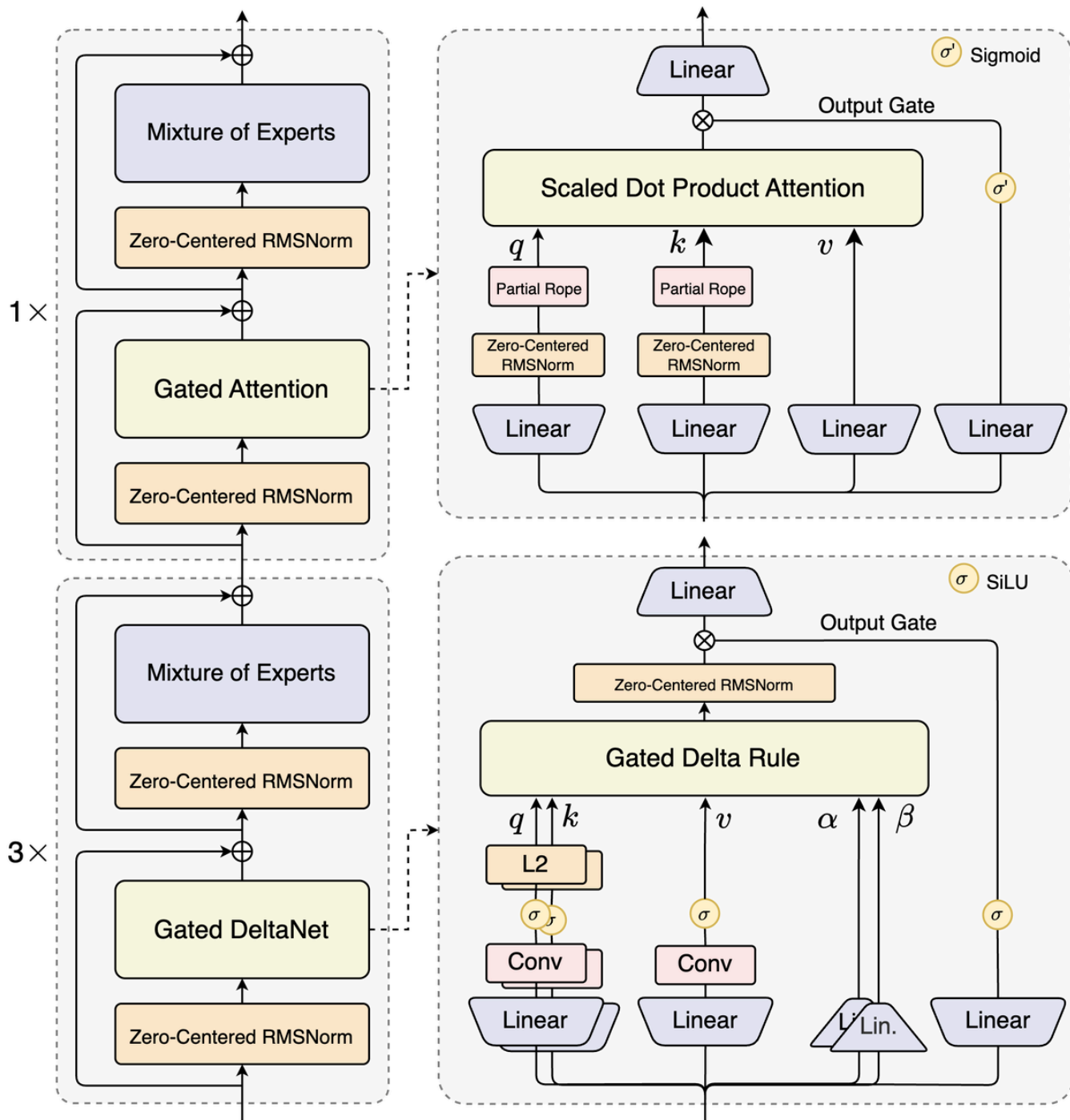
增加了更为完备的后训练，总体上仍然遵循SFT->RL的范式，但是细化了RL，具体参考 [Qwen3-VL](#)。

5. Qwen3.5

5.1 架构

Qwen-3.5 是一个 Dense 多模态大模型（当然也支持Text-Only模式）：

- 多模态架构上相比 [Qwen3-VL](#) 去除了复杂的 [DeepStack](#) 结构，回归了 [Qwen2-VL](#)/[Qwen2.5-VL](#) 的 ViT + PatchMerger + LLM 的三段式结构。
- LLM 架构上 Qwen-3.5 参考 [Qwen3 Next](#) 使用支持交错 Gated/Linear Attention 架构（每4层：3层 GatedDeltaNet + 1层 Gated Attention）
- 整体上看 Qwen-3.5 就是多模态版的 [Qwen3 Next](#)。



- ViT：架构回归了 [Qwen2-VL](#) / [Qwen2.5-VL](#) 的 [NaViT](#) 架构，不再使用 [Qwen3-VL](#) 的 [DeepStack](#)。
- Projector：Projector 沿用 [Qwen2-VL](#) / [Qwen2.5-VL](#) / [Qwen3-VL](#) 的 PatchMerger 结构。
- LLM：LLM 最重要的特点是交错 **Gated/Linear Attention** 架构（本质上参考的Qwen-Next），默认是每4层 Transformer Block中有3层是 **GatedDeltaNet** 有1层是 **Gated Attention**（**比例为3:1**）

5.2 训练

目前信息仅限于 [Qwen3.5官方Blog](#) 信息，主要改进集中于预训练视觉-文本数据、多语言数据的scaling、以及RL的 infra&scaling，待技术报告发布后详细更新。